

Terceiro Trabalho de Reconhecimento de Padrões

Jakson da Rocha Almeida

12 de junho de 2026

Introdução

Este relatório apresenta o terceiro trabalho de reconhecimento de padrões: classificação de risco de crédito com o banco Statlog (German Credit Data), disponível no repositório UCI. A implementação foi feita em um notebook Jupyter, em Python, com apoio de bibliotecas usuais de análise de dados e aprendizado de máquina.

O enunciado solicita seis classificadores — mínimos quadrados linear, vizinho mais próximo (kNN), rede MLP, SVM não-linear, AdaBoost e floresta aleatória — comparados por validação cruzada estratificada, com pré-processamento dos dados e avaliação conforme as aulas da disciplina.

Foi adotada semente fixa para reprodutibilidade e $k = 5$ partições na validação cruzada. As figuras deste relatório foram geradas a partir dos experimentos descritos no notebook.

Banco de dados

O conjunto contém $n = 1000$ pedidos de crédito, cada um com $l = 20$ atributos (7 numéricos e 13 categóricos) e uma classe binária:

- classe 1 (good): bom pagador;
- classe 2 (bad): mau pagador (risco de inadimplência).

No trabalho, a classe good foi codificada como 0 e a bad como 1, que é a classe positiva nas métricas de precisão e revocação. A distribuição é desbalanceada: aproximadamente 700 exemplos good (70%) e 300 bad (30%). A documentação do conjunto traz uma matriz de custos assimétrica: classificar um mau cliente como bom tem custo 5, enquanto o erro inverso tem custo 1. Por isso, além da acurácia, vale acompanhar a revocação da classe bad.

Utilizamos a versão com atributos categóricos em formato simbólico, o que exige codificação das variáveis categóricas antes do treinamento.

Pré-processamento

De acordo com a aula de Pré-processamento e Avaliação de Classificadores, foram realizadas as etapas abaixo:

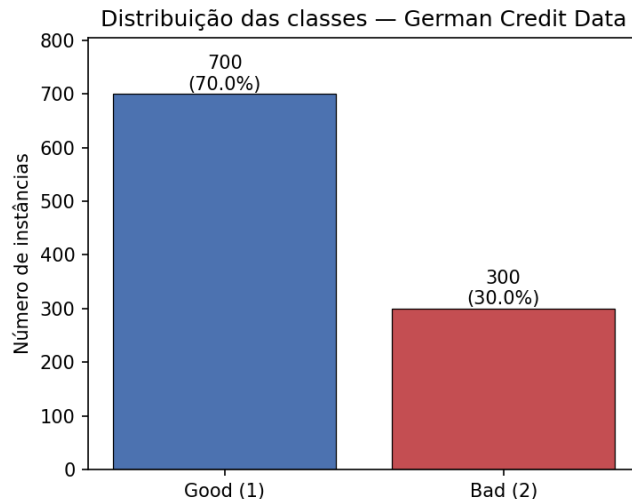


Figura 1: Distribuição das classes no German Credit Data (700 good, 300 bad).

1. Caracterização — 1000 instâncias e 20 atributos brutos; após a codificação categórica, o número de atributos passa a ser de aproximadamente 61.
2. Limpeza — não foram encontrados valores faltantes nem inconsistências evidentes.
3. Codificação — variáveis categóricas transformadas em representação binária (one-hot) para as 13 colunas categóricas.
4. Padronização — normalização z -score aplicada somente às 7 colunas numéricas, mantendo as demais sem alteração de escala.

A padronização é importante para kNN, SVM e MLP. No caso do MQ, também ajuda a trabalhar com atributos em escalas mais comparáveis após a expansão one-hot.

Balanceamento com SMOTE

Dado o desbalanceamento 70/30, adotamos o SMOTE (Synthetic Minority Over-sampling Technique) com $k = 5$ vizinhos, aplicado apenas ao conjunto de treino de cada fold da validação cruzada. O conjunto de teste de cada fold permanece com a distribuição original, para evitar vazamento de informação entre treino e teste.

Em todos os classificadores, a ordem das etapas foi a mesma: pré-processamento, SMOTE no treino e, por fim, o classificador escolhido. Sem balanceamento, os modelos tendem a favorecer a classe majoritária (good), o que reduz a revocação de bad.

Metodologia de avaliação

Utilizamos validação cruzada estratificada com $k = 5$ folds, de modo que cada partição de treino e teste preserve a proporção 70/30 entre as classes. As previsões de teste de cada fold foram reunidas para montar a matriz de confusão e a curva ROC.

As métricas seguem a aula de métricas para avaliação de classificadores, considerando bad (rótulo 1) como classe positiva:

- Acurácia — fração de acertos no total;
- Precisão — $VP/(VP+FP)$;
- Revocação (recall ou taxa de verdadeiros positivos) — $VP/(VP+FN)$;
- F1-score — média harmônica de precisão e revocação;
- AUC-ROC — área sob a curva ROC, calculada a partir de um escore contínuo de confiança do classificador.

Para cada métrica, reportamos a média e o desvio padrão entre os cinco folds.

Mínimos quadrados linear (MQ)

O classificador linear por mínimos quadrados foi implementado com matriz aumentada $\tilde{\mathbf{X}} = [\mathbf{1} \mid \mathbf{X}]$, alvos ± 1 e solução $\hat{\boldsymbol{\theta}} = \tilde{\mathbf{X}}^+ \mathbf{y}$ por pseudoinversa. A classe prevista é definida pelo sinal de $\tilde{\mathbf{x}}^\top \hat{\boldsymbol{\theta}}$.

Resultados (CV $k=5$, com SMOTE). Acurácia $0,714 \pm 0,022$; precisão $0,518 \pm 0,027$; revocação $0,703 \pm 0,032$; F1 $0,596 \pm 0,028$; AUC $0,776 \pm 0,015$. O MQ apresentou boa revocação de bad, embora com precisão moderada.

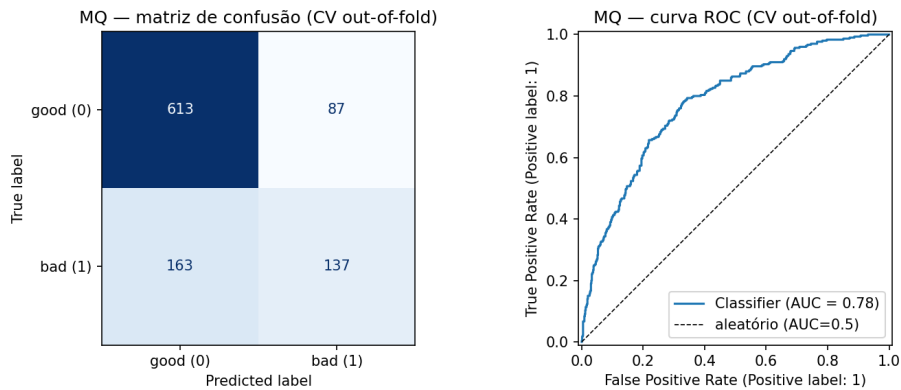


Figura 2: MQ: matriz de confusão e curva ROC na validação cruzada ($k=5$).

Vizinho mais próximo (kNN)

Foi utilizado o método do vizinho mais próximo com $k = 5$, o mesmo valor adotado no primeiro trabalho. Com o SMOTE, a revocação aumentou em relação ao caso sem balanceamento, com queda na acurácia.

Resultados (CV $k=5$, com SMOTE). Acurácia $0,630 \pm 0,024$; precisão $0,429 \pm 0,022$; revocação $0,703 \pm 0,041$; F1 $0,533 \pm 0,025$; AUC $0,701 \pm 0,021$.

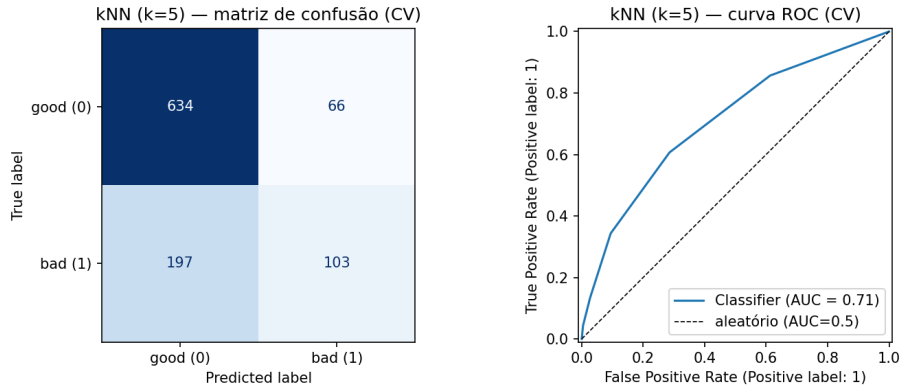


Figura 3: kNN ($k=5$): matriz de confusão e curva ROC na validação cruzada.

Rede MLP (Perceptron Multicamadas)

A rede possui duas camadas ocultas, com 64 e 32 neurónios, função de ativação ReLU e treinamento pelo método Adam. Foi usada parada antecipada para limitar sobreajuste.

Resultados (CV $k=5$, com SMOTE). Acurácia $0,710 \pm 0,027$; precisão $0,517 \pm 0,036$; revocação $0,660 \pm 0,061$; F1 $0,577 \pm 0,026$; AUC $0,762 \pm 0,026$.

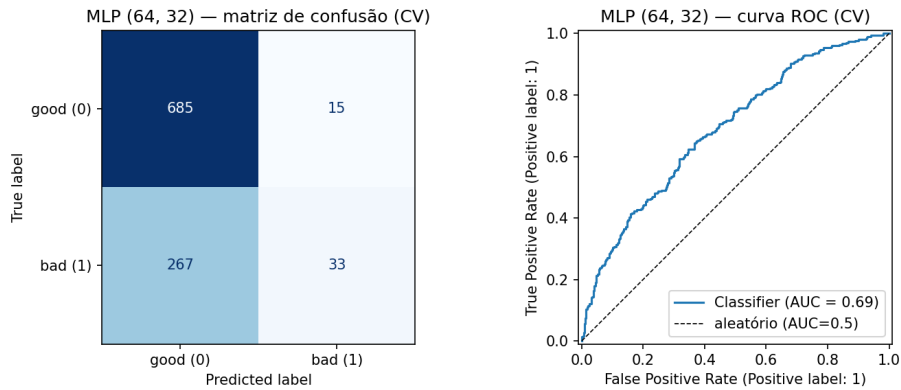


Figura 4: MLP com camadas (64,32): matriz de confusão e curva ROC na validação cruzada.

SVM não-linear (kernel RBF)

No segundo trabalho foi usado SVM linear; aqui adotamos kernel radial (RBF), com parâmetro de regularização $C = 1$, para permitir fronteiras de decisão não lineares entre os atributos de crédito.

Resultados (CV $k=5$, com SMOTE). Acurácia $0,736 \pm 0,021$; precisão $0,559 \pm 0,037$; revocação $0,573 \pm 0,062$; F1 $0,565 \pm 0,041$; AUC $0,780 \pm 0,018$. O SVM com kernel RBF manteve precisão e revocação em níveis intermediários, com AUC próximo dos melhores resultados observados.

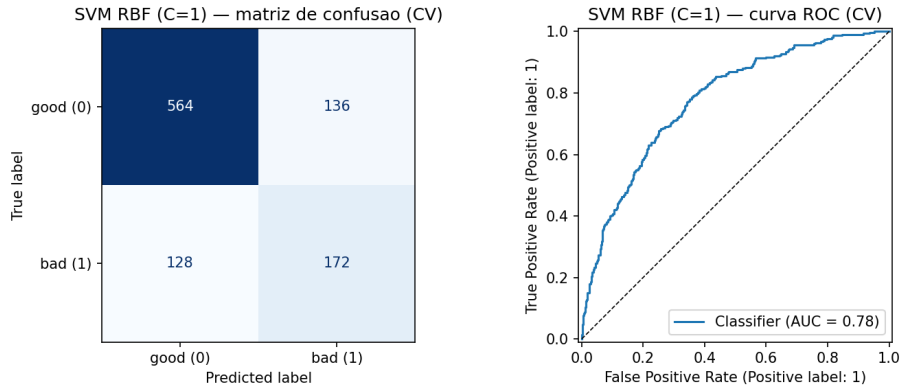


Figura 5: SVM com kernel RBF ($C=1$): matriz de confusão e curva ROC na validação cruzada.

AdaBoost

O AdaBoost foi configurado com 100 iterações, usando árvores de decisão de profundidade 1 como classificadores base.

Resultados (CV $k=5$, com SMOTE). Acurácia $0,735 \pm 0,017$; precisão $0,555 \pm 0,029$; revocação $0,600 \pm 0,033$; F1 $0,576 \pm 0,023$; AUC $0,778 \pm 0,018$. Os resultados ficaram próximos aos do SVM com kernel RBF.

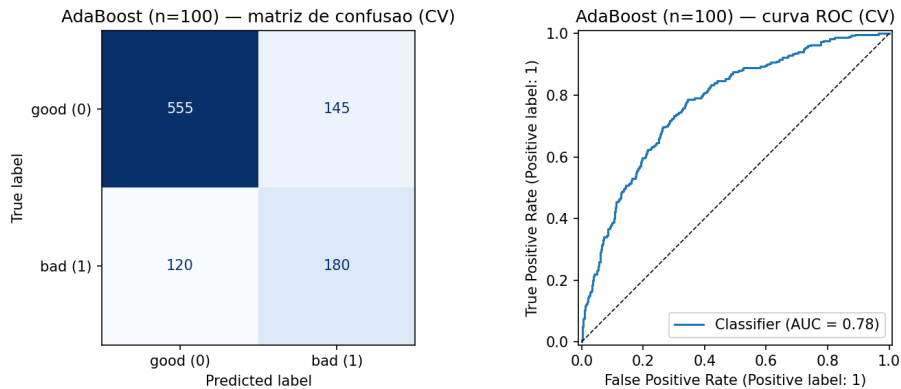


Figura 6: AdaBoost com 100 classificadores base: matriz de confusão e curva ROC na validação cruzada.

Floresta aleatória (Random Forest)

A floresta aleatória foi treinada com 100 árvores de decisão; a classe final é definida por votação majoritária.

Resultados (CV $k=5$, com SMOTE). Acurácia $0,765 \pm 0,009$; precisão $0,657 \pm 0,037$; revocação $0,460 \pm 0,017$; F1 $0,540 \pm 0,009$; AUC $0,784 \pm 0,028$. Entre os seis métodos, a floresta aleatória obteve a maior acurácia e a maior precisão, mas também a menor revocação, ou seja, tendeu a classificar menos clientes como bad.

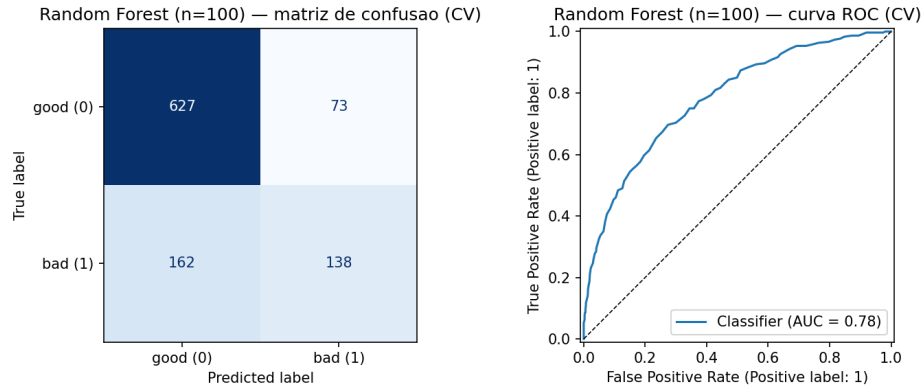


Figura 7: Floresta aleatória com 100 árvores: matriz de confusão e curva ROC na validação cruzada.

Comparação dos seis classificadores

Todos os métodos foram avaliados com o mesmo pré-processamento, o mesmo balanceamento por SMOTE no treino e a mesma validação cruzada estratificada com $k = 5$.

Classificador	Acc	Prec	Rec	F1	AUC	Custo
MQ (pseudoinversa)	0,714	0,518	0,703	0,596	0,776	642
kNN ($k=5$)	0,630	0,429	0,703	0,533	0,701	726
MLP (64, 32)	0,710	0,517	0,660	0,577	0,762	698
SVM RBF ($C=1$)	0,736	0,559	0,573	0,565	0,780	776
AdaBoost ($n=100$)	0,735	0,555	0,600	0,576	0,778	745
Floresta aleatória ($n=100$)	0,765	0,657	0,460	0,540	0,784	883

Tabela 1: Médias das métricas na validação cruzada ($k=5$), com SMOTE aplicado no treino de cada fold. A coluna Custo é o custo total agregado das predições ($5 \cdot \text{FN} + 1 \cdot \text{FP}$), conforme a matriz de custos da documentação.

Quando a comparação leva em conta a matriz de custos (falso negativo cinco vezes mais caro que falso positivo), o custo total inverte o ranking sugerido pela acurácia: o MQ apresenta o menor custo (642) e a floresta aleatória o maior (883), justamente por ter a menor revocação de bad.

Efeito do SMOTE

Para ilustrar a necessidade do balanceamento, compararam-se os casos com e sem SMOTE em MQ, kNN e MLP, na mesma validação cruzada:

- Sem SMOTE: a acurácia foi um pouco maior (por exemplo, cerca de 0,75 no MQ), mas a revocação de bad ficou baixa (cerca de 0,46 no MQ e 0,34 no kNN), pois o modelo passava a prever good com mais frequência.
- Com SMOTE: a revocação de bad subiu para cerca de 0,70 no MQ e no kNN, com F1 em torno de 0,60 no MQ; a acurácia diminuiu levemente, já que mais exemplos

passaram a ser classificados como bad, o que parece adequado diante da matriz de custos do problema.

Essa forma de aplicar o SMOTE apenas no treino de cada fold segue a orientação discutida na aula de Pré-processamento e Avaliação de Classificadores.

Conclusão

O pré-processamento — codificação das variáveis categóricas, padronização das numéricas e balanceamento no treino — foi necessário para comparar os seis classificadores em um conjunto desbalanceado.

Pela acurácia e pela precisão, a floresta aleatória apresentou os melhores valores (0,765 e 0,657, respectivamente). Pela revocação de bad, que é relevante por causa do custo maior de falsos negativos, os melhores resultados ficaram com MQ e kNN (cerca de 0,70), seguidos por MLP e AdaBoost. O AUC-ROC ficou em torno de 0,78 para MQ, SVM, AdaBoost e floresta aleatória, o que indica separação razoável entre as classes, independentemente do limiar de decisão adotado.

Não há um único classificador claramente superior em todas as métricas. A escolha depende do que se deseja priorizar: detectar mais clientes de risco (revocação) ou reduzir falsos alarmes (precisão), conforme a matriz de custos descrita na documentação do conjunto de dados. Levando em conta essa matriz de custos ($5 \cdot \text{FN} + 1 \cdot \text{FP}$), o MQ obteve o menor custo total e a floresta aleatória o maior, o que mostra que a maior acurácia não implica o melhor desempenho neste problema.